

Zero to Hero Study Handbook: hierarchical-qlora-complaint-triage

Module 1: Foundations & Architecture

1.1 What this project does

This repository implements a two-stage hierarchical complaint triage system for financial complaints:

1. Stage 1 predicts `l1` (product category).
2. Stage 2 predicts `l2` (issue category) conditioned on the chosen `l1`.

Main use cases from the code and README:

1. Train a local hierarchical router on CFPB + Banking77 with QLoRA (`make_notebook.py` , `trriage_model.py` , `trriage_worker.py`).
2. Run production-style routing with confidence and top-3 outputs (`inference.py`).
3. Compare baselines and monitor calibration/uncertainty (`trriage_baselines.py` , `calibration.py` , `active_learning.py`).

1.2 Core paradigms and patterns used here

Definitions first:

1. Hierarchical classification: predict a parent label (`l1`) and then a child label (`l2`) inside that parent.
2. Candidate-label scoring (not free generation): score each allowed label as a completion and normalize with softmax.
3. QLoRA fine-tuning: 4-bit quantized base model + small LoRA adapters for low-VRAM training.
4. Calibration: transform raw confidence so predicted probability better matches empirical accuracy.
5. Active learning: choose most uncertain unlabeled examples first to improve model efficiently.

Patterns visible in this repo:

1. Centralized config object pattern via `TriageConfig` and global `CONFIG` (`trriage_config.py`).
2. Functional pipeline style for data/model helpers (`trriage_data.py` , `trriage_model.py` , `calibration.py` , `active_learning.py`).
3. Thin OOP wrappers where state is useful: `HierScorer` in `trriage_model.py` , `TriageRouter` and `TriageResult` in `inference.py` .
4. Orchestrator + worker subprocess pattern: `make_notebook.py` launches `trriage_worker.py` subprocesses to avoid GPU memory accumulation between repeated trainings.

1.3 Architecture (components and interaction)

Key modules:

1. Data acquisition: `download_cfpb.py` fetches CFPB API pages and writes `data/raw/cfpb/cfpb_subset.parquet` .

2. Data curation and splits: `trriage_data.py` builds unified dataframe and hierarchy.
3. Modeling/scoring/evaluation: `trriage_model.py`.
4. Training orchestration: Notebook generated by `make_notebook.py`, plus subprocess worker `trriage_worker.py`.
5. Inference packaging: `inference.py`.
6. Supporting evaluation: `trriage_baselines.py`, `calibration.py`, `active_learning.py`.

Main flow (text ASCII diagram):

```

CFPB API --> download_cfpb.py --> data/raw/cfpb/cfpb_subset.parquet
      |
      v
      triage_data.build_dataframe()
      |
      +-----+-----+
      |               |
      v               v
hierarchy: dict[l1]->list[l2]  df(text,l1,l2,source)
      |               |
      +-----+-----+
      v
      make_splits(train,val,test,pool)
      |
      v
      triage_model.make_sft_records(train, hierarchy)
      |
      v
      SFTTrainer (QLoRA) in notebook/worker
      |
      v
      adapter_dir + hierarchy.json + metrics/extras
      |
      v
      inference.TriageRouter.route()/route_batch()
      -> score_l1 -> choose l1 -> score_l2(l1-specific)
      -> TriageResult(l1/l2/confidence/top3)

```

Module 2: Repository Map

File/Directory Path	Primary Responsibility	Key Classes/Functions	Important Configs/Variables
<code>README.md</code>	Project overview, dataset strategy, run commands, expected outputs	N/A	Run commands (<code>uv sync</code> , <code>download_cfpb.py</code> , <code>make_notebook.py</code> , <code>inference.py</code>)
<code>pyproject.toml</code>	Python package/dependency	N/A	<code>requires-python=">=3.12.10"</code> ,

File/Directory Path	Primary Responsibility	Key Classes/Functions	Important Configs/Variables
	manifest		dependencies, [tool.uv].torch-backend="cu128"
trriage_config.py	Single source of truth for model/data/training paths and knobs	TriageConfig, CONFIG	model_id, cfpb_parquet, split fractions, LoRA params, adapter_dir, figures_dir
download_cfpb.py	Fetch balanced CFPB subset via public API and save parquet	fetch_product, main	API, CANONICAL_PRODUCTS, FIELDS, OUT, CLI --per-product
trriage_data.py	Build curated dataset and hierarchy; create stratified splits	banking77_to_l1, clean_narrative, build_dataframe, make_splits, label_maps	Regex cleaners, min_l2_count, max_per_l2, train_cap, split keys train/val/test/pool
trriage_model.py	Prompt templates, SFT record creation, candidate scoring, evaluation, model loading	l1_system, l2_system, make_sft_records, HierScorer, hier_evaluate, load_base_model	L1_GLOSS, eval_batch_size, temperature args, 4-bit BitsAndBytesConfig
trriage_baselines.py	CPU baseline with TF-IDF + Logistic Regression	_pipe, BaselineResult, run_tfidf_baseline	TfidfVectorizer params, LogisticRegression(max_iter=2000, C=4.0, class_weight="balanced")
calibration.py	Calibration math and fitting helpers	softmax_T, expected_calibration_error, fit_temperature, fit_isotonic, reliability_points	n_bins, max_iter, torch LBFGS over logT
active_learning.py	Uncertainty scoring and pool selection	least_confidence, margin_uncertainty, entropy, select	_STRATEGIES, strategy, budget

File/Directory Path	Primary Responsibility	Key Classes/Functions	Important Configs/Variables
<code>triage_worker.py</code>	One-process train+eval worker for notebook subprocess calls	<code>main</code>	JSON config keys (<code>model_id</code> , <code>target_modules</code> , <code>train_parquet</code> , <code>bs</code> , <code>accum</code> , <code>epochs</code> , <code>eval_bs</code> , <code>train_cap</code> , <code>zero_shot</code> , <code>score_pool</code> , <code>pool_sample</code> , <code>out_dir</code> , <code>out_json</code>)
<code>inference.py</code>	User-facing hierarchical routing API/CLI	<code>load_hierarchy</code> , <code>TriageResult</code> , <code>TriageRouter</code> , <code>main</code>	CLI flags <code>--text</code> , <code>--base-only</code> ; <code>temperature_ll</code> ; <code>hierarchy.json</code> loading
<code>make_notebook.py</code>	Programmatically generate full tutorial notebook	<code>md</code> , <code>code</code> (cell builders); <code>embedded_free_trainer</code> , <code>finetune</code> , <code>route_one</code> , <code>run_worker</code> , <code>al_curve</code> , <code>bake</code> ; writes notebook	<code>OUT="hierarchical_qlora_complaint_triage.ipynb"</code> , <code>QUICK_MODE=False</code> in generated notebook
<code>hierarchical_qlora_complaint_triage.ipynb</code>	End-to-end executable workflow artifact	Notebook cells mirror module functions and subprocess orchestration	Uses <code>CONFIG</code> , writes figures, saves adapter/hierarchy
<code>figures/</code>	Saved plots for evaluation analysis	N/A	<code>data_overview.png</code> , <code>model_comparison.png</code> , <code>calibration.png</code> , <code>active_learning.png</code> , <code>ll_confusion.png</code> , <code>backbone_bakeoff.png</code>

Module 3: Core Execution Flows

Flow A: Data acquisition (`download_cfpb.py`)

Step-by-step:

- `main()` parses `--per-product` (default `2000`).
- For each canonical product in `CANONICAL_PRODUCTS`, it calls `fetch_product(raws, target)`.
- `fetch_product` loops through raw CFPB product strings and paginates API requests with params: `product`, `has_narrative=true`, `size`, `from`, `sort=created_date_desc`, `no_aggs=true`, `repeated` field.
- Response rows are reduced to `FIELDS`:
`["product", "issue", "sub_issue", "complaint_what_happened", "date_received", "company"]`.
- `main()` adds `product_ll` per row, de-duplicates on `complaint_what_happened`, and writes parquet to `OUT`.

Input/output shape:

1. `fetch_product(...)` -> `list[dict]` where each dict contains `FIELDS`.
2. Output parquet columns: `product`, `issue`, `sub_issue`, `complaint_what_happened`, `date_received`, `company`, `product_l1`.

Short real fragment:

```
rows.append({k: src.get(k) for k in FIELDS})
row["product_l1"] = canon
df.to_parquet(OUT, index=False)
```

Flow B: Curation + hierarchy + split (`trriage_data.py`)

Step-by-step:

1. `build_dataframe(cfg)` validates `cfg.cfpb_parquet` exists.
2. Reads CFPB parquet, renames columns: `product_l1` -> `l1`, `issue` -> `l2`, `complaint_what_happened` -> `text`.
3. Cleans narratives with `clean_narrative`: `XXXX` redactions become `[redacted]`, whitespace collapsed, near-empty redacted narratives dropped, max char truncation applied.
4. Filters out short text using `cfg.min_chars`.
5. Applies anti-sparsity and balancing: drop `(l1,l2)` classes with `count < cfg.min_l2_count`; cap each issue at `cfg.max_per_l2`.
6. Optional Banking77 augmentation: `banking77_to_l1(intent)` maps intent to one of three L1 products; `l2=None`; `source="banking77"`.
7. Concatenates CFPB + Banking77, drops duplicate `text`.
8. Builds hierarchy: `dict[str, list[str]]` from CFPB rows only and ensures all L1 keys exist.
9. `make_splits(df, cfg)` creates stratified splits by `l1` into keys: `train`, `val`, `test`, `pool`; then caps `train` to `cfg.train_cap`.

Input/output shape:

1. Curated dataframe columns are exactly: `text` (str), `l1` (str), `l2` (str or None), `source` ("cfpb" or "banking77").
2. `build_dataframe(...)` -> `tuple[pd.DataFrame, dict[str, list[str]]]`.
3. `make_splits(...)` -> `dict[str, pd.DataFrame]` with keys `train`, `val`, `test`, `pool`.
4. `label_maps(hierarchy)` returns: `l1_labels`, `l2_labels`, `l1_to_id`, `l2_to_id`.

Flow C: Training + scoring + evaluation (`trriage_model.py`, `notebook`, `worker`)

Step-by-step:

1. Prompt builders: `l1_messages(text, l1_labels)` and `l2_messages(text, l1, l2_candidates)`.
2. `make_sft_records(train_df, hierarchy, tokenizer, cfg)` creates mixed records: stage-1 record for every row; stage-2 only when `row.l2` is valid under `row.l1`.
3. Each SFT record has shape: `{"prompt": <chat-template-string>, "completion": <label-string>}`.
4. Notebook/worker instantiates `SFTTrainer` with LoRA config and completion-only loss.
5. `HierScorer` scores candidate completions using token log-likelihood averages.

6. `hier_evaluate(...)` computes metrics: `l1_macro_f1`, `l1_accuracy`, `l1_top3_accuracy`, `l2_macro_f1`, `l2_accuracy`, `hier_exact_match`.
7. `trriage_worker.py` isolates repeated fits in subprocesses and returns JSON metrics to orchestrators.

Input/output shape details:

1. `HierScorer.score_l1(texts, l1_labels)` returns `list[dict[str, float]]` where dict maps each L1 label to probability.
2. `HierScorer.score_l2(texts, l1s, hierarchy)` returns `list[dict[str, float]]` over issue set per provided `l1`.
3. `hier_evaluate` returns: `(metrics_dict, extras_dict)`.
4. `extras_dict` keys are: `l1_probs`, `pred_l1`, `gold_l1`, `l1_labels`, `l1_score_matrix`.
5. Worker output JSON includes: `zero_shot` (optional), `fine_tuned`, `peak_gb`, and when pool scoring is enabled also `pool_idx`, `pool_l1_probs`.

Short real fragment:

```
records.append({"prompt": p1, "completion": row.l1})
if row.l2 is not None and row.l2 in hierarchy.get(row.l1, []):
    records.append({"prompt": p2, "completion": row.l2})
```

Flow D: Inference router (`inference.py`)

Step-by-step:

1. `TriageRouter.__init__` loads hierarchy from `<adapter_dir>/hierarchy.json` if present, else rebuilds via `build_dataframe()`.
2. Loads base model optionally with LoRA adapter through `load_base_model(...)`.
3. `route_batch(texts)` runs stage-1 probabilities for all texts.
4. Picks predicted `l1` via `argmax`.
5. Runs stage-2 only for rows where `self.hierarchy[pred_l1]` is non-empty.
6. Builds `TriageResult` with confidence and top-3 for each stage.

`TriageResult` exact fields:

1. `l1: str`
2. `l1_confidence: float`
3. `l1_top3: list[tuple[str, float]]`
4. `l2: str | None`
5. `l2_confidence: float | None`
6. `l2_top3: list[tuple[str, float]] | None`

CLI paths:

1. `--text "..."` routes one complaint.
2. No `--text` runs built-in `_DEMOS`.
3. `--base-only` disables adapter attachment (zero-shot mode).

Module 4: Setup & Run Guide

4.1 Machine prerequisites

1. Python `>=3.12.10` (from `pyproject.toml`).
2. `uv` package manager.
3. For training/inference paths that load 4-bit models, CUDA GPU is required by notebook assertion.
4. For CPU-only inspection scripts (`trriage_data.py` , `trriage_baselines.py` , `calibration.py`), GPU is not required.

4.2 Install dependencies

```
uv sync
```

Dependency source of truth: `pyproject.toml` and `uv.lock`.

4.3 Data setup

```
uv run python download_cfpb.py --per-product 2000
```

Expected artifact: `data/raw/cfpb/cfpb_subset.parquet`.

4.4 Generate and use the notebook workflow

```
uv run python make_notebook.py
uv run jupyter lab hierarchical_qlora_complaint_triage.ipynb
```

Notes from static code inspection:

1. Generated notebook sets `QUICK_MODE = False` in `make_notebook.py`.
2. Notebook sets runtime env defaults internally:
`PYTORCH_CUDA_ALLOC_CONF=expandable_segments:True` and `TOKENIZERS_PARALLELISM=false`.

4.5 CLI routing after model artifacts exist

```
uv run python inference.py --text "A debt collector keeps calling about a paid loan"
uv run python inference.py
uv run python inference.py --base-only
```

`inference.py` expects adapter artifacts under `CONFIG.adapter_dir` (`outputs/qlora-triage-adapter`) and reads `hierarchy.json` there when present.

4.6 Auxiliary scripts

```
uv run python triage_data.py
uv run python triage_baselines.py
uv run python calibration.py
```

4.7 Environment variables and migrations

1. `.env` keys: none required by repository code.
2. External services: CFPB public API and Hugging Face dataset/model downloads are used by scripts.
3. Database migrations/seeding: none found; no database layer is present.

4.8 PDF export (for this handbook)

```
pandoc ZERO_TO_HERO_STUDY_HANDBOOK.md -o ZERO_TO_HERO_STUDY_HANDBOOK.pdf
```

Module 5: Study Plan & Practice Exercises

5.1 Ordered study plan

1. Read `README.md` and `pyproject.toml` for project intent, constraints, and dependencies.
2. Read `trriage_config.py` to understand all shared knobs and paths.
3. Read `download_cfpb.py` then `trriage_data.py` for data lineage and split logic.
4. Read `trriage_model.py` deeply: prompt construction, `make_sft_records`, `HierScorer`, and `hier_evaluate`.
5. Read `inference.py` to understand production-facing route behavior and return schema.
6. Read `trriage_worker.py` and `make_notebook.py` to understand orchestration and repeated training design.
7. Read `trriage_baselines.py`, `calibration.py`, and `active_learning.py` for quality-control subsystems.
8. Open `hierarchical_qlora_complaint_trriage.ipynb` to map each notebook section back to module functions.

5.2 Practice exercises (with solution outlines)

1. Exercise: Explain why stage-2 scoring only uses issues under one `l1`. Solution outline: In `TriageRouter.route_batch`, predicted `l1` is computed first; stage-2 candidates come from `self.hierarchy[l1]`, reducing label space and aligning with product-specific issues.
2. Exercise: Trace the exact dataframe columns from raw CFPB parquet to curated train split. Solution outline: `download_cfpb.py` writes raw fields + `product_l1`; `build_dataframe` renames to `text/l1/l2`, adds `source`, filters, merges Banking77; `make_splits` returns `train/val/test/pool` dataframes with same columns.
3. Exercise: Identify where probability calibration is fitted vs applied. Solution outline: `calibration.fit_temperature` fits on validation score matrix (`val_S`, `val_y` in notebook), then `softmax_T(test_S, T)` applies to test set.
4. Exercise: Describe how this repo avoids invalid label outputs from LLM generation. Solution outline: `HierScorer` scores fixed candidate labels as completions and softmaxes them; no free-form decoding path is used for label decisions.
5. Exercise: List all metrics returned by `hier_evaluate` and explain which rows each uses. Solution outline: `l1_*` metrics use all rows; `l2_*` and `hier_exact_match` use CFPB rows with valid L2 under gold L1, via mask logic in `hier_evaluate`.

6. Exercise: Explain why `trriage_worker.py` exists when notebook already has training code. Solution outline: Worker isolates each fit in a subprocess so OS fully reclaims GPU memory between runs; notebook comments mention in-process VRAM leakage risk.
7. Exercise: Compare `run_tfidf_baseline` L2 setup with hierarchical L2 setup. Solution outline: Baseline L2 is flat over all CFPB issues (`l2.predict(te2["text"])`); hierarchical model conditions L2 candidate set on `l1` via `score_l2`.
8. Exercise: Reconstruct worker config schema by reading code only. Solution outline: From `trriage_worker.py` docstring and `main`, required/used keys include `model_id`, `target_modules`, `train_parquet`, `bs`, `accum`, `epochs`, `eval_bs`, `train_cap`, `zero_shot`, `score_pool`, `pool_sample`, `test_sample`, `out_dir`, `out_json`.
9. Exercise: Explain fallback behavior when `hierarchy.json` is missing at inference time. Solution outline: `load_hierarchy(adapter_dir)` checks for file first; if missing it calls `build_dataframe()` and rebuilds hierarchy from current data.
10. Exercise: Find one intentional reproducibility control and one potential source of drift. Solution outline: Reproducibility: `set_seed(CONFIG.seed)` and split randomness seeded. Potential drift: hierarchy rebuild fallback depends on current local data snapshot if adapter file missing.

Learner Verification Checklist

1. Can you explain the end-to-end route from CFPB API rows to `TriageResult` fields?
2. Can you describe why `build_dataframe` drops sparse (`l1,l2`) classes and caps per-issue counts?
3. Can you explain exactly how `make_sft_records` mixes stage-1 and stage-2 supervision?
4. Can you walk through how `HierScorer` turns candidate label strings into probabilities?
5. Can you distinguish `l2_macro_f1` from `hier_exact_match` in `hier_evaluate`?
6. Can you list all keys in `TriageResult` and when `l2` is `None`?
7. Can you explain how `trriage_worker.py` and notebook subprocess orchestration prevent GPU-memory accumulation?
8. Can you identify all high-impact knobs in `TriageConfig` for data balance, model capacity, and training runtime?
9. Can you explain where calibration enters the pipeline and how it affects confidence-gated routing?
10. Can you run mentally through how active-learning `select(...)` chooses rows for labeling under different strategies?